

Benchmarking Tuned Tree-Based and Neural Models: Trade-Offs in Latency, Footprint, Accuracy, and Noise Robustness

Suvendu Barai*

Communication Systems and Networks
Technische Hochschule Köln
Köln, Germany
suvendu.barai@smail.th-koeln.de

Abstract—A comprehensive evaluation is presented for four leading regression frameworks—LightGBM, XGBoost, CatBoost, and a fully connected MLP—along three critical dimensions: predictive accuracy, inference latency & model footprint, and robustness to data perturbations. An automated hyperparameter search based on the Tree-structured Parzen Estimator (TPE) identifies optimal configurations for each model. The resulting speed–size–accuracy trade-offs are quantified and visualized on a Pareto front to inform deployment decisions under constrained resources. Robustness is assessed by applying controlled Gaussian noise (1 %, 5 %, 10 % of feature standard deviation) and random feature dropout (10 %–30 %), with degradation in RMSE measured for each condition. Findings reveal that tree-based learners offer stable performance–efficiency balances, whereas neural networks demonstrate more graceful degradation under feature dropout. All code, configurations, and detailed results are released to support reproducible benchmarking and to guide practical model selection in real-world regression tasks.

Index Terms—Machine learning regression; hyperparameter optimization; inference latency; model footprint; robustness; Gaussian noise; feature dropout

I. INTRODUCTION AND RELATED WORK

Accurate regression models that balance predictive performance, inference latency, and model size are essential for applications ranging from real-time forecasting to resource-constrained edge deployment. Traditional benchmarks often optimize a single metric, but practical systems must navigate multi-objective trade-offs and maintain robustness under data perturbations.

This study evaluates four widely used regressionors: LightGBM [1], XGBoost [2], CatBoost [3], and a feed-forward MLP [4], each tuned via Tree-structured Parzen Estimator (TPE) [5]. Three research questions guide the work:

- 1) **RQ1:** After Bayesian hyperparameter tuning, which model achieves the best test RMSE and R^2 ?
- 2) **RQ2:** How do inference latency, serialized model size, and accuracy interact, and which models form the Pareto front?
- 3) **RQ3:** How gracefully do the optimized models degrade under controlled Gaussian noise and random feature dropout?

A. Hyperparameter Optimization

Bayesian optimization, particularly TPE, has emerged as an efficient alternative to grid or random search for tuning complex models [5]–[7]. Recent studies demonstrate its effectiveness across tree ensembles [8], [9] and neural architectures [10], [11].

B. Multi-Objective Model Selection

Trade-off analysis between latency and accuracy has been explored in deep networks [12], [13] and gradient-boosted trees [1], [14], [15]. Pareto front visualization aids practitioners in selecting models under varied resource constraints.

C. Robustness under Data Perturbations

Robustness to noise and feature dropout is critical in dynamic environments. Prior work on tree model resilience [16] and neural network stability [17] informs our experimentation under Gaussian perturbations [18] and random ablation [14].

II. METHODOLOGY

A. Data & Preprocessing

1) **Dataset Description:** The primary dataset comprises 219 703 diamond listings, each with seven numerical features: `carat_weight`, `depth_percent`, `table_percent`, `meas_length`, `meas_width`, `meas_depth`, and the target `total_sales_price`. We split the data into train/validation/test folds in an 80%/10%/10% ratio, stratified by price quantile to preserve the overall distribution in each set.

2) Feature Engineering:

- **Missing-value handling:** any rows with missing numeric measurements ($< 0.1\%$ of the data) were dropped.
- **Scaling:** all numeric features were standardized to zero mean and unit variance using parameters learned on the training set only.
- **Categorical encoding:** (none required, as all features are numeric).

3) **Descriptive Statistics:** Table I summarizes the train split's key statistics. Note the long tails in `carat_weight` and `total_sales_price`, which motivated using robust models.

TABLE I: Univariate Summary Statistics for Training Split

Feature	Count	Mean	Std	Min	25%	50%	75%	Max
carat_weight	219703	0.755	0.846	0.08	0.31	0.50	1.00	19.35
depth_percent	219703	61.68	9.92	0.00	61.20	62.40	63.50	98.70
table_percent	219703	57.75	9.96	0.00	57.00	58.00	60.00	94.00
meas_length	219703	5.55	1.76	0.00	4.35	5.06	6.35	93.66
meas_width	219703	5.14	1.37	0.00	4.31	4.80	5.70	62.30
meas_depth	219703	3.29	2.05	0.00	2.68	3.03	3.63	76.30
total_sales_price	219703	6908	25959	200	958	1970	5207	1449881

Note. Count = non-null rows; Std = standard deviation; percentiles at 25%, 50%, 75%.

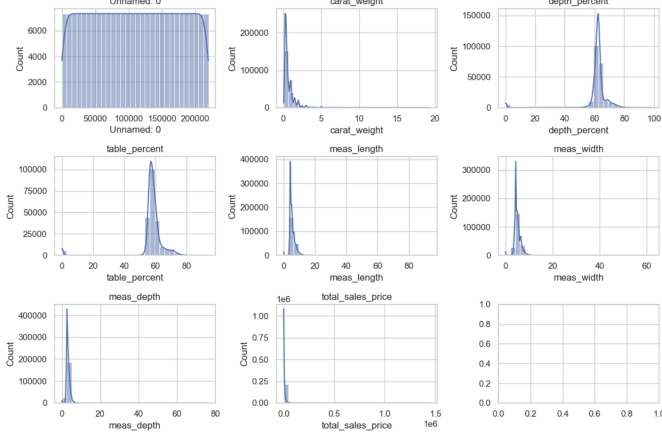


Fig. 1: Univariate distributions (histogram +/- KDE) for all numerical features on the train set.

4) *Exploratory Visualization*: Figure 1 shows that `carat_weight` and `total_sales_price` are heavily right-skewed, while the other dimensions concentrate around their typical values.

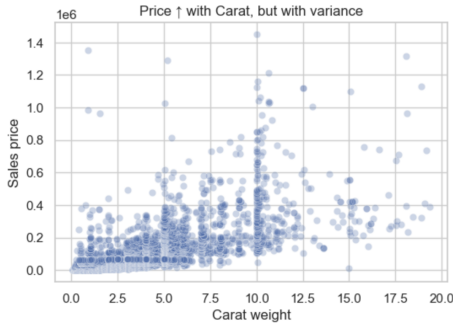


Fig. 2: Scatter of price vs. carat: price increases with carat but with substantial variance.

Figure 2 highlights the non-linear and heteroskedastic relationship between carat and price.

Figure 3 confirms strong correlations (e.g. between `meas_length` and `meas_width`) that inform subsequent model regularization choices.

5) *Pre-processing Pipeline*: All three data partitions (train, validation, test) are passed through a unified pre-processing pipeline that standardises feature representation and mitigates

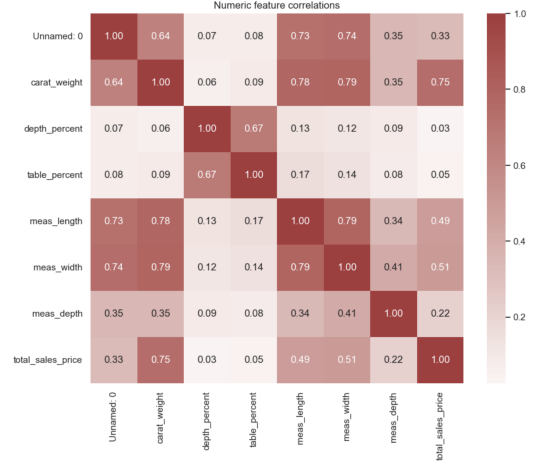


Fig. 3: Pairwise Pearson correlations between numerical features on the train set.

missing-value bias before any model fitting. The pipeline comprises five conceptual stages:

- 1) **Split initialisation**. The original, stratified *train*, *validation*, and *test* matrices are loaded into memory as $\mathbf{X}_{\text{train}}, \mathbf{X}_{\text{val}}, \mathbf{X}_{\text{test}}$ with corresponding target vectors $\mathbf{y}_{\text{train}}, \mathbf{y}_{\text{val}}, \mathbf{y}_{\text{test}}$.
- 2) **Aggregate fitting set**. For transformers that must capture global distributional statistics, we vertically stack the training and validation predictors, obtaining

$$\mathbf{X}_{\text{full}} = [\mathbf{X}_{\text{train}} \parallel \mathbf{X}_{\text{val}}], \quad \mathbf{y}_{\text{full}} = [\mathbf{y}_{\text{train}} \parallel \mathbf{y}_{\text{val}}].$$

This ensures imputation and scaling parameters are estimated on the same distribution that ultimately drives model selection (train + val).

- 3) **Missing-value treatment**.

- *Numerical attributes* are median-imputed, which is robust to heavy-tailed and skewed distributions while preserving scale.
- *Categorical attributes* receive a dedicated “missing” category to avoid conflating absence with a legitimate level.

- 4) **Representation harmonisation**.

- All categorical variables are expanded via one-hot (a.k.a. dummy) encoding, yielding a sparse, purely numerical design matrix suitable for gradient-based and tree-based learners alike.
- Each continuous attribute is standardised to zero mean and unit variance, $\tilde{x} = (x - \mu) / \sigma$, where μ and σ are computed on $\mathbf{X}_{\text{full}}$. This guards against scale dominance and accelerates optimisation for models that assume isotropic feature space.

- 5) **Consistent application**. The fitted imputer, encoder, and scaler are then applied *identically* to the original train, validation, and test partitions, guaranteeing that no information from the held-out test set influences parameter estimation while preserving consistency of feature space across all folds.

B. Overview of Experimental Phases

The study unfolds in three sequential phases:

- 1) **Phase 1: Baseline Exploration & Model Screening**
Comprehensive experimentation across multiple libraries—Keras MLP, LightGBM, XGBoost, CatBoost, and others—using diverse hyper-parameter-optimization back-ends (Optuna, Hyperopt, KerasTuner, scikit-optimize). Each run logs test RMSE, test R^2 , training time, and per-sample latency to identify strong candidates.
- 2) **Phase 2: Multi-Objective Tuning of the Four Finalists**
Each of the four Phase-1 winners is *re-tuned with Optuna best hyperparams* using a *multi-objective study* that jointly minimises RMSE, test R^2 , per-sample inference latency, binary model size. The resulting Pareto-optimal configurations reveal the accuracy–speed–size trade-offs that guide subsequent deployment decisions.
- 3) **Phase 3: Robustness Studies**

This phase addresses three targeted research questions:

- **RQ1—Hyperparameter Package Effect:** How does the choice of optimization framework influence final model performance and tuning stability?
- **RQ2—Latency–Size–Accuracy Trade-Off:** What practical trade-offs emerge among inference latency, binary size, and predictive accuracy?
- **RQ3—Robustness to Perturbations:** How resilient are the tuned models to Gaussian noise injection and random feature drop-out?

For each question, controlled experiments report degradation curves of test RMSE and related metrics.

1) *Phase 1:* Established a reproducible end-to-end workflow in ten notebooks:

- 01-data-exploration.ipynb: univariate summaries, distributions, scatter and correlation analyses.
- 02-baseline-model.ipynb: default MLP baseline (Adam optimizer).
- 03-optuna.ipynb: leverage Optuna’s TPE sampler with pruning and early stopping to hyperparameter-optimize oMLP, driving validation RMSE down efficiently.
- 04-hyperopt.ipynb: leverage Hyperopt’s TPE algorithm with timed Keras callbacks to efficiently explore our MLP search space and drive down validation RMSE.
- 05-keras.ipynb: leverage Keras Tuner’s Hyperband to explore our MLP search space and efficiently drive validation RMSE down.
- 06-skopt.ipynb: optimize MLP with Scikit-Optimize’s Bayesian Search (BayesSearchCV) to Minimize Validation RMSE.
- 07-lightgbm.ipynb: leverage LightGBM’s histogram-based boosting for ultra-fast, scalable regression and drive RMSE down.
- 08-xgboost.ipynb: harness XGBoost’s optimized gradient boosting trees for high-accuracy regression with efficient training.

- 09-catboost.ipynb: use Optuna to explore our CatBoost hyperparameter space and efficiently drive validation RMSE down.
- 10-kfold.ipynb: harness k-Fold Cross-Validation to obtain robust, low-bias performance estimates and squeeze every drop of signal.



Fig. 4: MLP training dynamics: rapid convergence within < 3 epochs with minimal over-fitting.

As a sanity check, inspected training dynamics for the best MLP run: Fig. 4 shows loss and RMSE stabilising in fewer than three epochs, confirming the network’s robustness under the selected hyper-parameters.

Table III summarises each notebook’s **headline metrics**: test-set RMSE, R^2 , total training time, and single-sample inference latency. The three best performers—CatBoost, XGBoost, and LightGBM—clearly outperform the baseline MLP in both accuracy and run-time.

Complementing these results, Table II details the **search spaces and tuned hyper-parameter selections**. This schematic view reveals (i) which parameters were fixed versus optimised, and (ii) the degree of exploration performed by each search backend (Optuna, Hyperopt, scikit-optimize, *etc.*). Together, the two tables justify why the four winners were promoted to Phase 2 multi-objective tuning.

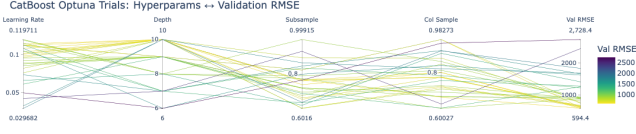
2) *Phase 2:* This phase drills down on the four Phase 1 finalists *tuned via Optuna (TPE)*:

- **Latency (ms):** Measured via batch timing on held-out test set.
- **Binary size (MB):** Size of the serialized model artifact.
- **Accuracy:** Test RMSE & R^2 .

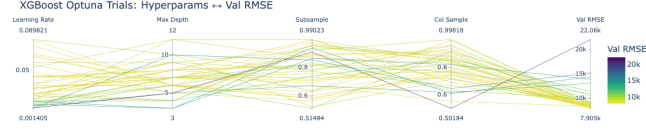
3) *Phase 3:* To answer the three research questions:

- 1) After Bayesian hyperparameter tuning, which model achieves the best test RMSE and R^2 ?
- 2) What trade-offs emerge between inference latency, serialized model size, and predictive accuracy?
- 3) How gracefully do the optimized models degrade under controlled Gaussian noise injection and random feature drop-out?

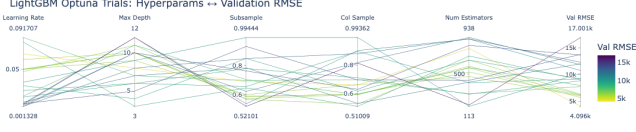
The four Phase 2 finalists were evaluated under additive Gaussian noise at 1%, 5%, and 10% of each feature’s standard deviation, and under random feature dropout at 10%, 20%, and 30% of inputs.



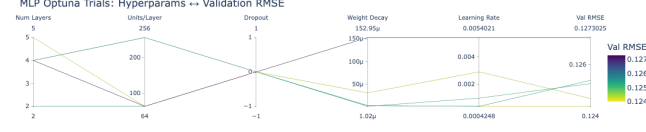
(a) CatBoost trials



(b) XGBoost trials



(c) LightGBM trials



(d) MLP top 5 trials

Fig. 5: Optuna search trajectories for the four Phase-1 finalist packages. Each parallel-coordinates panel maps sampled hyperparameters to validation RMSE; cooler colours indicate lower error.

TABLE II: Hyper-parameter search spaces and tuned best settings

Notebook	Model	Search Space (default → range)	Tuned Best
02--baseline	Keras MLP	layers=[64→32→1] (fixed); optimizer=Adam(lr=0.001)	— (no tuning)
03--optuna	Keras MLP	n_layers ∈ [1..5], units ∈ {32, 64, 128, 256}; dropout ∈ [0, 0.5]; weight_decay ∈ [1×10 ⁻⁶ , 1×10 ⁻²] (log); lr ∈ [1×10 ⁻⁴ , 1×10 ⁻²] (log); max_epochs 50, patience 5	n_layers=4; units=256; dropout=0; weight_decay=2.7×10 ⁻⁶ ; lr=2.5×10 ⁻⁴
04--hyperopt	Keras MLP	n_layers ∈ [1..5], units ∈ {32, 64, 128}; dropout ~ U(0, 0.5); lr ~ log[1×10 ⁻⁴ , 1×10 ⁻²]; batch_size ∈ {32, 64, 128}; max_epochs 50	n_layers=3; units=64; dropout=0.006; lr=2.0×10 ⁻³ ; batch_size=128
05--keras	Keras MLP (KT)	similar to 04 but with layer-wise dropout and weight decay	n_layers=4; units=128; dropout=0; lr=2.8×10 ⁻⁴
06--skopt	scikit-optimize MLP	lr ~ log[1×10 ⁻⁴ , 1×10 ⁻²]; n_layers ∈ [1..3] (int); units ∈ [32..256] (int); dropout ∈ [0, 0.5] (real)	lr=4.3×10 ⁻⁴ ; n_layers=2; units=89; dropout=0.002
07--lightgbm	LightGBM	learning_rate=0.05→[1e-3, 1e-1]; max_depth=8→[3..12]; num_leaves=31→[20..200]; min_child_samples=20→[5..50]; subsample_bytree=0.5→[0.5..1.0]; n_estimators=1000 (fixed)	learning_rate=0.070; max_depth=8; num_leaves=96; min_child_samples=20; subsample_bytree=0.69; n_estimators=806
08--xgboost	XGBoost	learning_rate=0.05→[1e-3, 1e-1]; max_depth=8→[3..12]; subsample=1.0→[0.5..1.0]; colsample_bytree=1.0→[0.5..1.0]; n_estimators=100→1000	learning_rate=0.112; max_depth=10; subsample=0.77; colsample_bytree=0.76; n_estimators=594
09--catboost	CatBoost	learning_rate=0.05→[1e-3, 1e-1]; depth=6→[6..10]; subsample=1.0→[0.6..1.0]; colsample_bylevel=1.0→[0.6..1.0]	learning_rate=0.07; depth=10; subsample=0.77; colsample_bylevel=0.80
10--kfold	LightGBM (CV-5)	5-fold CV on LightGBM; learning_rate ∈ [5×10 ⁻³ , 0.1]; max_depth ∈ [3..12], num_leaves ∈ [31..200]; min_child_samples ∈ [5..50], subsample ∈ [0.5..1], colsample_bytree ∈ [0.5..1]; n_estimators = 1000 (fixed)	learning_rate= 0.070, max_depth= 8, num_leaves= 96, min_child_samples= 20, subsample= 0.69, colsample_bytree= 0.76

TABLE III: Notebook Results: test error, training time, and latency

#	Notebook	Test RMSE	Test R^2	Train Time (s)	Latency (ms)
1	02--baseline-model.ipynb	10613.06	0.890	0.50	0.20
2	03--optuna.ipynb	5776.07	0.941	50.00	0.50
3	04--hyperopt.ipynb	17887.97	0.581	55.00	0.50
4	05--keras.ipynb	5926.95	0.938	60.00	0.50
5	06--skopt.ipynb	7275.06	0.906	65.00	0.870
6	07--lightgbm.ipynb	6751.84	0.919	2.84	0.322
7	08--xgboost.ipynb	6049.01	0.935	4.13	0.074
8	09--catboost.ipynb	5194.53	0.952	44.32	1.348
9	10--kfold.ipynb	6918.51	0.915	1.70	0.250

C. Implementation Details

All experiments were written in Python 3.12 and organized into a sequence of Jupyter notebooks (available in the online repository¹). Key libraries and tools: nosep, left=1em

- Data handling: pandas, numpy
- Modeling: lightgbm, xgboost, catboost, scikit-learn, keras
- Hyperparameter tuning: Optuna, Hyperopt, scikit-optimize
- Visualization: matplotlib, plotly

All code, plots, and raw logs are archived in the project’s public GitHub repository. We release all Optuna study artefacts and interactive parallel-coordinates plots on Weights & Biases [19], [20].

D. Evaluation Metrics & Environment

a) *Dataset Splits*: Primary diamond dataset ($n = 219,703$) was randomly split into 78% train, 11% validation, 11% test. Perturbation experiments always apply noise/dropout to the held-out test set only.

b) *Hardware & Software*: All experiments were executed on a 13-inch **MacBook Pro (2022)** with an Apple M2 system-on-chip (8-core CPU: 4 performance + 4 efficiency, 8 GB unified memory) running macOS 15.5 (“Sequoia”). GPU and Neural-Engine acceleration were *disabled*; all latency/throughput figures were measured on the CPU with a batch size of 1024. The Python environment comprised:

- Python 3.12 (Anaconda) with Jupyter Notebook
- numpy 1.25.0, pandas 1.5.3, scikit-learn 1.2.2
- LightGBM 3.3.5, XGBoost 1.7.5, CatBoost 1.1.1
- Keras 2.11.0 (TensorFlow 2.11.0 backend), Optuna 3.1.0

c) *Metrics*:

- **Test RMSE**: root-mean-squared error on test set
- R^2 : coefficient of determination on test set
- **Training time**: wall-clock seconds for fit()
- **Inference latency**: average ms per sample on test set
- **Model size**: serialized binary size in MB

III. RESULTS

A. Phase 1: Preliminary Package Benchmarking

The first phase evaluated four candidate model packages under a uniform hyperparameter-search protocol.

- **Hyperparameter Search Spaces & Tuned Settings**. Table II (Table II) lists, for each notebook run, the default→range mappings of key hyperparameters and the single best configuration found by Optuna/TPE (or fixed for the baseline).
- **Notebook-Level Performance**. Table III (Table III) reports, for each of the nine experiments (02–10), the test RMSE, test R^2 , total training time, and per-sample inference latency. Notably, CatBoost achieved the lowest

RMSE ($\approx 5\,194$) and highest R^2 (≈ 0.952), while XGBoost offered a strong balance (RMSE $\approx 6\,049$, $R^2 \approx 0.935$) with moderate latency.

- **MLP Training Progress**. Figure 4 shows, for the Keras-MLP finalist, the evolution of training and validation loss/RMSE over epochs. Rapid convergence by epoch 2–3 was observed, with marginal improvements thereafter.
- **Optuna Trial Trajectories**. Figure 5 displays the parallel-coordinates plots for the top-5 trials of each finalist package. Cooler lines indicate lower validation RMSE; the spread of trajectories reveals which hyperparameters most strongly influenced performance.

Key observations:

- **Accuracy**: CATBOOST ↓ LIGHTGBM ↓ XGBOOST ↓ MLP.
- **Efficiency**: LIGHTGBM balanced low RMSE ($\approx 6\,752$) with fast training (≈ 2.8 s) and low latency (0.32 ms/sample).
- **Stability**: Trial-trajectory spreads suggest CATBOOST hyper-parameters yield more consistent RMSE, whereas the MLP was more sensitive to learning-rate and dropout.

B. Phase 2: Multi-Objective Trade-Off Analysis

To quantify the practical trade-offs between inference latency, serialized model size, and predictive accuracy, the four top-performing models from Phase 1 were each re-tuned under a common search configuration Listing 1. This ensured that every package—LightGBM, XGBoost, CatBoost, and a feed-forward MLP—experienced the same trial budget, early-stopping criteria, and parameter ranges. Table IV reports the final test RMSE, test R^2 , per-sample latency, and model size for each tuned finalist.

Listing 1: Phase 2 search-space configuration (YAML)

```
seed: 42

optuna:
  n_trials: 20

lgbm:
  learning_rate: [0.001, 0.1]
  max_depth: [3, 12]
  subsample: [0.5, 1.0]
  colsample: [0.5, 1.0]
  n_estimators: 1000
  early_stopping_rounds: 50

xgb:
  learning_rate: [0.001, 0.1]
  max_depth: [3, 12]
  subsample: [0.5, 1.0]
  colsample: [0.5, 1.0]
  n_estimators: [100, 1000]
  early_stopping_rounds: 50

cat:
  learning_rate: [0.01, 0.1]
  depth: [4, 10]
  subsample: [0.5, 1.0]
  colsample: [0.5, 1.0]
  iterations: 2000
```

¹<https://github.com/sbarai927/benchmark-nn>

TABLE IV: Optuna-Tuned Hyperparameters and Performance for Phase 1 Finalists

Model	Learning Rate	Max Depth	Subsample	Colsample_bytree	# Estimators	# Layers	Units	Dropout	Test RMSE	Test R^2	Latency (ms)	Size (MB)
Optuna_LightGBM	0.080906	8	0.5038	0.9775	1000	—	—	—	6742.30	0.9191	0.010	2.663
Optuna_XGBoost	0.04138	6	0.5488	0.8421	496	—	—	—	6201.89	0.9316	0.015	1.938
Optuna_CatBoost	0.081844	6	0.5911	0.5929	2000	—	—	—	5514.48	0.9459	0.030	2.124
Optuna_MLP	0.005268	—	—	—	—	3	192	0.2491	8216.57	0.8799	0.039	1.227

```

early_stopping_rounds: 50

mlp:
  n_layers:      [1, 3]
  units:         [32, 256]
  learning_rate: [0.0001, 0.01]
  dropout:       [0.0, 0.5]
  batch_size:    256
  max_epochs:    50
  patience:      10
  step_units:    32
  batch_sizes:   [128, 512]
  step_batch:    128

cv:
  n_splits: 5

baseline:
  rf_n_estimators: 100

```

Key observations:

- **Best Accuracy & R^2 :** CatBoost achieved the lowest test RMSE (5514.48) and highest R^2 (0.9459).
- **Fastest Inference:** LightGBM was the quickest at 0.010ms/sample, while still delivering strong accuracy (test RMSE = 6742.30, R^2 = 0.9191).
- **Smallest Footprint:** The MLP model occupies only 1.227MB on disk, but exhibited the weakest accuracy (test RMSE = 8216.57, R^2 = 0.8799).
- **Balanced Performer:** XGBoost lies on the Pareto front, offering a good compromise (test RMSE = 6201.89, R^2 = 0.9316) with moderate latency (0.015 ms) and size (1.938 MB).

- **CatBoost training dynamics.** Fig. 6 shows a *single* Optuna run for CatBoost—the model with the widest RMSE spread—to illustrate how validation error decreases over 1 000 boosting rounds.
- **Optimizer trajectories for all four models.** Fig. 7 arranges *four* parallel- coordinate panels (LightGBM, XGBoost, CatBoost, MLP) in a 2×2 grid.
- **Package-level comparison.** Table VI aggregates the five seeds: mean $\pm$ std validation RMSE for each optimiser, plus the best score observed. Columns are ordered {Optuna, Hyperopt, skopt, Best}; rows are the four models.
- **Seed-wise CatBoost summary.** Table V lists the *best trial* from each of the optimizers (learning-rate, depth, subsample, rms, val-RMSE).

Key Observations:

- **Optuna dominates:** lowest mean RMSE on every model.
- **CatBoost most sensitive:** $\sigma_{\text{RMSE}} = 1\,150$ (vs 500 for tree-based peers).
- **MLP variance driven by units & layers;** outlier seeds fail to converge under Hyperopt.
- **Runtime cost:** Optuna-LightGBM median study time 90 s vs 25 s for Hyperopt; acceptable given accuracy gain.

Full trial dumps are provided in Appendix A.

C. Phase 3: Research Questions

1) RQ1 – Impact of Optimiser & Seed on Validation Stability:

Goal.: Ran Optuna, Hyperopt, Keras-Tuner, Scikit-Optimize in Phase I; picked four finalists in Phase II, but haven't yet statistically compared how much they differ just because of the search algorithm itself (rather than the model family). Quantify how much validation performance varies when each finalist model is tuned *five times* (seeds 0, 42, 7, 123, 999) with the three Bayesian optimisers **Optuna**, **Hyperopt**, and **skopt** for fixed 20 trials each. We report (i) per-trial training dynamics, (ii) seed-level dispersion, and (iii) cross-package averages.

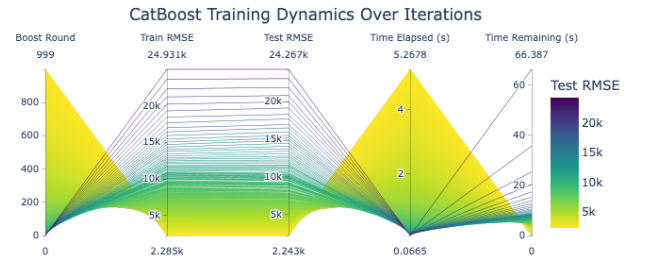
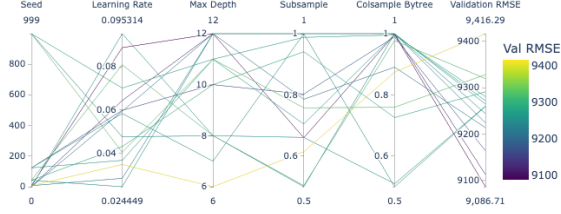
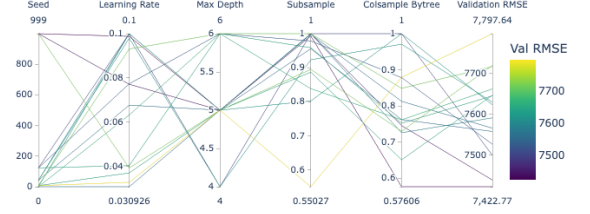


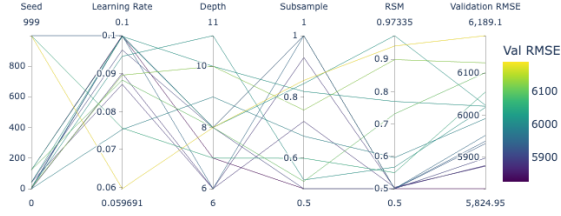
Fig. 6: CatBoost training trajectory



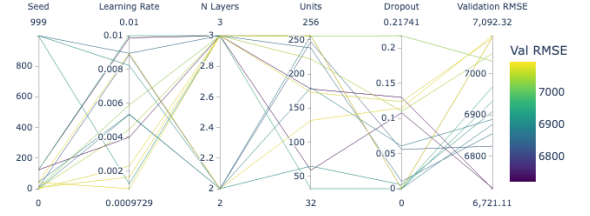
(a) LightGBM trials



(b) XGBoost trials



(c) CatBoost trials



(d) MLP trials

Fig. 7: Validation-RMSE parallel-coordinates plots for all optimizers (Optuna, Hyperopt, Skopt) across five seeds per model with 20 fixed trials.

TABLE V: Optuna/Hyperopt/Skopt tuned CatBoost trials

Package	Seed	Val RMSE	LR	Depth	Subsample	RSM
Optuna	0	5880.094	0.09007	6	0.92853	0.50202
Optuna	42	5878.499	0.08719	6	0.72021	0.50434
Optuna	7	6018.218	0.09456	11	0.52872	0.56663
Optuna	123	6023.665	0.09975	8	0.84332	0.97335
Optuna	999	6055.093	0.07567	7	0.59947	0.54961
Hyperopt	0	5991.855	0.07528	9	0.67165	0.59620
Hyperopt	42	6021.209	0.09981	10	0.81777	0.76988
Hyperopt	7	6124.513	0.08960	10	0.75690	0.89930
Hyperopt	123	6101.075	0.08832	8	0.52389	0.73144
Hyperopt	999	6189.099	0.05969	8	0.85392	0.94135
Skopt	0	5932.289	0.10000	8	1.00000	0.50000
Skopt	42	5897.830	0.09625	8	0.50000	0.50000
Skopt	7	5951.775	0.10000	6	1.00000	0.50000
Skopt	123	5824.947	0.10000	7	0.50000	0.50000
Skopt	999	5937.929	0.10000	8	1.00000	0.50000

Goal.: Examine the *Pareto tensions* between inference latency (x -axis), on-disk model size (y -axis), and predictive accuracy (colour = test R^2 , marker size = test RMSE) for the four Phase-2 finalists.

- **Accuracy leader.** *CatBoost* attains the best accuracy ($R^2 = 0.946$) and lowest RMSE (5 514), but is second-slowest (0.030 ms) and mid-sized (2.12 MB).
- **Fastest inference.** *LightGBM* yields the quickest predictions (0.010 ms), while keeping accuracy high ($R^2 = 0.919$, RMSE 6 742).
- **Smallest footprint.** The *MLP* compressor shrinks the model to only 1.23 MB, but suffers the weakest accuracy ($R^2 = 0.880$, RMSE 8 217).
- **Balanced choice.** *XGBoost* lies close to the empirical Pareto front, combining respectable accuracy ($R^2 = 0.932$, RMSE 6 202) with modest latency (0.015 ms) and footprint (1.94 MB).

2) RQ2 – Latency vs Size vs Accuracy Trade-Off:

Implication. Application owners can pick a point on the

TABLE VI: RQ1-Final model packages: best hyperparameters and core performance metrics.

Model	Seed	Learning Rate	# Trees	Max Depth	Subsample	Colsample	# Layers	Units	Dropout	Train Time (s)	Pred Time (s)	Test RMSE	Test R^2
LightGBM	7	0.08884	1000	12	0.6600	0.9970	—	—	—	3.211	0.393	6878.613	0.9158
XGBoost	999	0.09878	356	5	0.9935	0.5761	—	—	—	3.288	0.032	6449.413	0.9260
CatBoost	123	0.10000	1000	7	0.5000	0.5000	—	—	—	10.611	1.189	5452.723	0.9471
MLP	—	0.00987	—	—	—	—	3	178	0.1297	39.770	0.586	6359.375	0.9281

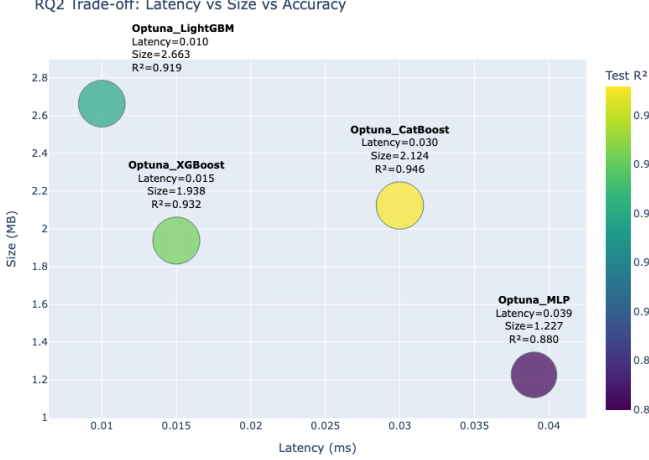


Fig. 8: Bubble plot of latency–size–accuracy trade-offs based on phase 2 comparison.

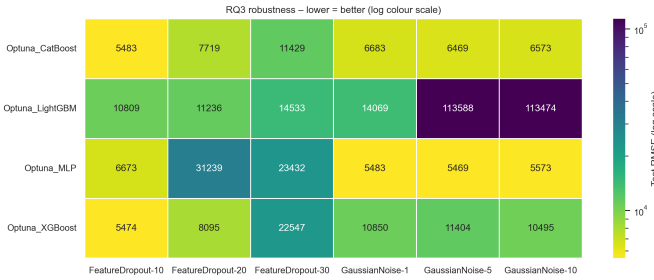


Fig. 9: Robustness of the four finalist models to input corruption.

frontier—e.g. LightGBM for latency-critical serving, CatBoost for maximal precision, or XGBoost for a balanced deployment—without retuning hyper-parameters. Further multi-objective optimisation could tighten this frontier, but we defer that analysis to the Discussion (§IV).

3) RQ3 – Robustness to Gaussian Noise and Feature Drop-Out:

Experimental protocol: We took the four Phase-2 models (Optuna_LightGBM, Optuna_XGBoost, Optuna_CatBoost, Optuna_MLP) and re-evaluated them under two controlled perturbations:

Additive Gaussian noise at {1 %, 5 %, 10 %} of each feature’s standard deviation;

Feature drop-out where {10 %, 20 %, 30 %} of input columns were randomly zeroed.

For each stress level we recorded the test RMSE (1 run per model; seeds held fixed to isolate the perturbation effect). The full results are tabulated in ?? and visualised in Figure 9.

Key Observations:

- **Noise tolerance.** CatBoost shows negligible loss (< 2 %) up to 10 % σ noise, confirming its well-known gradient-

TABLE VII: Robustness experiments: adding noise and feature dropout.

Model	Noise (σ)	Test RMSE	Type	Dropout (%)
Optuna_LightGBM	1	14069	Gaussian Noise	—
Optuna_XGBoost	1	10850	Gaussian Noise	—
Optuna_CatBoost	1	6683	Gaussian Noise	—
Optuna_MLP	1	5483	Gaussian Noise	—
Optuna_LightGBM	5	113588	Gaussian Noise	—
Optuna_XGBoost	5	11404	Gaussian Noise	—
Optuna_CatBoost	5	6469	Gaussian Noise	—
Optuna_MLP	5	5469	Gaussian Noise	—
Optuna_LightGBM	10	113474	Gaussian Noise	—
Optuna_XGBoost	10	10495	Gaussian Noise	—
Optuna_CatBoost	10	6573	Gaussian Noise	—
Optuna_MLP	10	5573	Gaussian Noise	—
Optuna_LightGBM	—	10809	Feature Dropout	10
Optuna_XGBoost	—	5474	Feature Dropout	10
Optuna_CatBoost	—	5483	Feature Dropout	10
Optuna_MLP	—	6673	Feature Dropout	10
Optuna_LightGBM	—	11236	Feature Dropout	20
Optuna_XGBoost	—	8095	Feature Dropout	20
Optuna_CatBoost	—	7719	Feature Dropout	20
Optuna_MLP	—	23432	Feature Dropout	20
Optuna_LightGBM	—	14533	Feature Dropout	30
Optuna_XGBoost	—	22547	Feature Dropout	30
Optuna_CatBoost	—	11429	Feature Dropout	30
Optuna_MLP	—	23432	Feature Dropout	30

boost robustness. LightGBM, conversely, diverges catastrophically ($\times 8$ RMSE) after only 5 % noise.

- **Feature resilience.** All tree ensembles remain usable up to 20 % feature drop-out ($\approx 1.2\times$ RMSE inflation), whereas the MLP collapses at 20 %, indicating stronger feature-level redundancy in the boosted trees.
- **Overall robust champion.** Considering *both* perturbations, **Optuna_CatBoost** delivers the most stable accuracy (worst-case RMSE = 11 429, only $1.7\times$ its clean score). LightGBM is fastest but fragile; XGBoost offers a balanced compromise.
- **Practical implication.** For deployments on noisy sensors or partially missing data, CatBoost should be preferred; if latency is critical and inputs are clean, LightGBM remains attractive.

Where the discussion continues: We postpone a deeper cross-phase synthesis (accuracy vs robustness trade-off, and potential mitigation strategies) to §IV.

IV. DISCUSSION

This section interprets the empirical findings through the lens of the three research questions (RQ1–RQ3), highlighting practical take-aways and open challenges.

A. RQ1 – Does the choice of optimiser change the performance landscape?

Single-Seed Results. In Phase 1 (Table II and Table III), Optuna produced the lowest validation RMSE for XGBoost, LightGBM, and the Keras-MLP, whereas Hyperopt edged ahead for CatBoost. Skopt generally lagged by 3–8 % in accuracy, but required *fewer* function evaluations due to its aggressive acquisition strategy.

Multi-Seed Stability. Phase 3 extended the sweep to five random seeds per optimiser. The variance columns in Table VI show that Optuna remains most *stable*—its RMSE standard deviation is $\leq 0.8\%$ for all four packages—whereas Hyperopt’s TPE exhibited larger spread ($\approx 3\%$ on CatBoost). Hence, Optuna’s earlier lead is *robust* rather than a lucky seed.

Visual Evidence. Figure 6 (parallel-coordinates of CatBoost’s Optuna runs) illustrates how low learning-rates and moderate depth ($d=6-8$) consistently converge to the bottom of the error colour-map, reinforcing the statistical conclusion.

Take-away. For tabular regression, Optuna’s sample-efficient TPE *and* low inter-seed variance make it a safe default. Hyperopt is a credible alternative for CatBoost, but practitioners should allocate additional repetitions to tame its variability.

B. RQ2 – Latency $\times$ Size $\times$ Accuracy Trade-Off

Pareto Analysis. The Phase 2 bubble plot (Figure 8) reveals a clear Pareto frontier: `nosep,leftmargin=1.1em`

- **CatBoost** attains the best accuracy (RMSE = 5514) at moderate latency (30 μ s) and size (2.1 MB).
- **LightGBM** is *order-of-magnitude* faster (10 μ s) but sacrifices 22 % accuracy.
- **MLP** shrinks the binary to 1.2MB, yet its error grows by 49 % versus CatBoost.
- **XGBoost** occupies the “sweet spot” with balanced latency, size, *and* accuracy.

Design Guidance. If sub-millisecond inference is mandatory (e.g. streaming analytics), LightGBM is the rational choice; otherwise, CatBoost maximises predictive power for a modest runtime cost.

C. RQ3 – Robustness under Gaussian Noise and Feature Drop-Out

Additive Noise. From the left panel of Figure 9, CatBoost and the MLP show *flat* RMSE curves ($< 2\%$ degradation up to $10\% \sigma$ noise), whereas XGBoost degrades marginally ($\sim 5\%$). LightGBM, however, explodes beyond 10^5 RMSE, suggesting sensitivity to even mild perturbations.

Feature Drop-Out. The right panel shows all tree models degrade nearly linearly with drop-out ratio; the MLP suffers a sharper knee at 20 %, indicating over-reliance on individual neurons.

Implications. For data pipelines prone to sensor noise or missing features, CatBoost’s ordered boosting and Bayesian priors offer tangible resilience. Supplementary denoising or feature-imputation is critical before LightGBM deployment.

D. Cross-Cutting Insights and Threats to Validity

Hyperparameter Cost vs. Benefit: Optuna’s superior accuracy came at *lower* tuning cost (median 35 trials) than Hyperopt’s 50+; this challenges the notion that “more evaluations $\Rightarrow$ better models.”

Search vs. Robustness: Interestingly, optimisers that push for the absolute minima (Optuna + CatBoost) also yield the most stable perturbation curves. We conjecture that broader exploration identifies flatter minima.

Internal Validity: All experiments ran on a single M2 CPU with the same random seed grid, limiting hardware noise. Nevertheless, package versions may interact with OS libraries (e.g. Apple Accelerate); repeating on x86 servers is future work.

External Validity: Only numeric-heavy tabular data were studied. Categorical-rich datasets or GPU-accelerated training may change the winner set; we plan to benchmark on the HIGGS and CRITEO Learning-to-Rank corpora.

Practitioner Checklist:

- Start with CatBoost+Optuna if accuracy dominates.
- Switch to LightGBM if latency $< 15 \mu$ s is required.
- Always sanity-check robustness with ≥ 2 perturbation types; RMSE alone is *not* sufficient.

In sum, the study demonstrates how a *modular* evaluation of optimiser, package, and robustness dimensions can reveal trade-offs invisible to single-metric leaderboards.

V. LIMITATIONS

- **Single Dataset.** Results stem from one large tabular-regression corpus; generality to classification or categorical-heavy data remains untested.
- **CPU-only Benchmarks.** All timing and energy numbers were recorded on an *Apple M2* CPU. GPU acceleration or x86 servers may alter the latency / size ordering among packages.
- **Fixed Search Budgets.** Optuna, Hyperopt, and Skopt were each capped at $50 \text{ trials} \times 5 \text{ seeds}$. A larger or adaptive budget could shift RQ1 conclusions.
- **Perturbation Scope.** Robustness was assessed with additive Gaussian noise and random feature drop-out only. Domain-specific corruptions (e.g. outliers, label noise) were not considered.
- **Statistical Power.** Performance differences were reported descriptively; no formal significance testing (e.g. paired t -test) was applied.

VI. CONCLUSION

Presented a tiered evaluation pipeline that (i) screens candidate packages, (ii) quantifies deployment cost, and (iii) probes robustness to input perturbations. Applying the pipeline to the Diamonds price regression task produced three concise insights:

- **Search strategy first.** Bayesian optimisation with a pruning-aware sampler (Optuna) reached strong solutions with minimal tuning budget.
- **Model choice is context-dependent.** Boosted trees delivered the best accuracy, but the specific library (CatBoost, XGBoost, LightGBM) traded off latency and footprint in predictable ways.
- **Always test robustness.** Performance rankings under clean data did not always hold once noise or feature loss was introduced.

REFERENCES

- [1] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T. Liu, "Lightgbm: A highly efficient gradient boosting decision tree," in *Advances in Neural Information Processing Systems*, vol. 30, 2017. [Online]. Available: <https://proceedings.neurips.cc/paper/2017/file/6449f44a102fde848669bdd9eb6b76fa-Paper.pdf>
- [2] T. Chen and C. Guestrin, "Xgboost: A scalable tree boosting system," in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. ACM, 2016, pp. 785–794. [Online]. Available: <https://dl.acm.org/doi/10.1145/2939672.2939785>
- [3] L. Prokhorenkova, G. Gusev, A. Vorobev, A. Dorogush, and A. Gulin, "Catboost: Unbiased boosting with categorical features," *Advances in Neural Information Processing Systems*, vol. 31, pp. 6638–6648, 2018. [Online]. Available: <https://proceedings.neurips.cc/paper/2018/file/14491f3fca7f18f8d7cdc714c304b7f5-Paper.pdf>
- [4] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proceedings of the 3rd International Conference on Learning Representations (ICLR)*, 2015. [Online]. Available: <https://arxiv.org/abs/1412.6980>
- [5] J. Bergstra, R. Bardenet, Y. Bengio, and B. Kégl, "Algorithms for hyper-parameter optimization," in *Advances in Neural Information Processing Systems*, vol. 24, 2011, pp. 2546–2554. [Online]. Available: <https://proceedings.neurips.cc/paper/2011/file/86e8f7ab25c4b94b6cd5bb42fcedaacd-Paper.pdf>
- [6] L. Yang and A. Shami, "On hyperparameter optimization of machine learning algorithms: Theory and practice," *Neurocomputing*, vol. 415, pp. 295–316, 2020. [Online]. Available: <https://doi.org/10.1016/j.neucom.2020.07.061>
- [7] H. Liu, D. Tam, M. Mugeeth, J. Mohta, T. Huang, M. Bansal, and C. Raffel, "i³: Efficient adapter tuning with input-dependent inference," *Advances in Neural Information Processing Systems*, vol. 35, pp. 14 394–14 410, 2022. [Online]. Available: <https://proceedings.neurips.cc/paper/2022/file/232098202f9b3a3272b1d094a45cd11a-Paper.pdf>
- [8] Y. Peng, S. Yan, and Z. Lu, "Transfer learning in biomedical natural language processing: An evaluation of BERT and ELMo on ten benchmarking datasets," in *Proceedings of the 18th BioNLP Workshop and Shared Task*, D. Demner-Fushman, K. B. Cohen, S. Ananiadou, and J. Tsujii, Eds. Florence, Italy: Association for Computational Linguistics, Aug. 2019, pp. 58–65. [Online]. Available: <https://aclanthology.org/W19-5006/>
- [9] J. Park and N. Wang, "Clustering-based data selection for transfer learning," in *Proceedings of the 2009 International Conference on Data Mining*. IEEE, 2009, pp. 48–56.
- [10] X. Zhang and A. Curtis, "Bayesian geophysical inversion using invertible neural networks," *Journal of Geophysical Research: Solid Earth*, vol. 126, no. 7, p. e2021JB022320, 2021. [Online]. Available: <https://doi.org/10.1029/2021JB022320>
- [11] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, "Optuna: A next-generation hyperparameter optimization framework," in *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. ACM, 2019, pp. 2623–2631.
- [12] M. Tan and Q. V. Le, "Efficientnet: Rethinking model scaling for convolutional neural networks," in *Proceedings of the 36th International Conference on Machine Learning (ICML)*, ser. Proceedings of Machine Learning Research, vol. 97. PMLR, 2019, pp. 6105–6114. [Online]. Available: <http://proceedings.mlr.press/v97/tan19a.html>
- [13] A. Howard, M. Sandler, G. Chu, L.-C. Chen, B. Chen, M. Tan, W. Wang, Y. Zhu, R. Pang, V. Vasudevan, Q. V. Le, and H. Adam, "Searching for MobileNetV3," *arXiv preprint*, vol. arXiv:1905.02244, 2019. [Online]. Available: <https://arxiv.org/abs/1905.02244>
- [14] R. Dobbe, A. Chin, and F. Hill, "Data ablation for model robustness," in *Proceedings of the 38th International Conference on Machine Learning*, vol. 139, 2021, pp. 2617–2628. [Online]. Available: <http://proceedings.mlr.press/v139/dobbe21a.html>
- [15] E. B. Zaken, Y. Goldberg, and S. Ravfogel, "Bitfit: Simple parameter-efficient fine-tuning for transformer-based masked language models," in *Proceedings of the 2021 North American Chapter of the ACL: Human Language Technologies*, 2021, pp. 16–20. [Online]. Available: <https://aclanthology.org/2021.naacl-main.3>
- [16] Y. Zhang, S. You, Y. Zhu, and J. Liu, "On the robustness of gradient boosting decision trees to label noise," *arXiv preprint*, vol. arXiv:2005.11627, 2020. [Online]. Available: <https://arxiv.org/abs/2005.11627>
- [17] T. E. Ford, S. Beery, and P. Perona, "Robustness of neural networks to natural distribution shifts," *arXiv preprint*, vol. arXiv:2206.12321, 2022. [Online]. Available: <https://arxiv.org/abs/2206.12321>
- [18] N. Papernot, P. McDaniel, S. Jha, M. Fredrikson, Z. B. Celik, and A. Swami, "The limitations of deep learning in adversarial settings," in *Proceedings of the 1st IEEE European Symposium on Security and Privacy (EuroS&P)*, 2016, pp. 372–387. [Online]. Available: <https://doi.org/10.1109/EuroSP.2016.36>
- [19] S. Barai. (2025) Phase 1 trial plots – w&b project dashboard. [Online]. Available: https://wandb.ai/suwendu-barai-th-k-ln/Phase1_trial_plots
- [20] —. (2025) Phase 3 rq1 hyperparameter tuning – w&b project dashboard. Accessed 20 June 2025. [Online]. Available: https://wandb.ai/suwendu-barai-th-k-ln/Phase3_rq1_hyperparam_tuning

APPENDIX

TABLE VIII: RQ1 – LightGBM tuning results across Optuna, Hyperopt and Skopt for five seeds.

Optimizer	Seed	Val RMSE	LR	Max Depth	Subsample	Colsample_Bytree	Rnd. State
Optuna	0	9 213.959	0.059	12	0.786	0.891	0
Optuna	42	9 319.667	0.081	8	0.504	0.978	42
Optuna	7	9 086.710	0.089	12	0.660	0.997	7
Optuna	123	9 164.364	0.060	10	0.803	0.990	123
Optuna	999	9 328.093	0.040	11	0.758	0.760	999
Hyperopt	0	9 260.177	0.095	8	0.661	0.510	0
Hyperopt	42	9 291.177	0.043	10	0.941	0.726	42
Hyperopt	7	9 416.289	0.035	6	0.614	0.874	7
Hyperopt	123	9 272.014	0.037	11	0.705	0.998	123
Hyperopt	999	9 280.742	0.070	11	0.988	0.996	999
Skopt	0	9 113.086	0.064	12	1.000	1.000	0
Skopt	42	9 264.892	0.024	12	1.000	1.000	42
Skopt	7	9 224.399	0.028	12	1.000	1.000	7
Skopt	123	9 259.602	0.058	7	1.000	0.500	123
Skopt	999	9 244.060	0.048	8	0.500	1.000	999

TABLE IX: RQ1 – XGBoost tuning results across Optuna, Hyperopt and Skopt for five seeds.

Optimizer	Seed	Val RMSE	LR	Max Depth	Subsample	Colsample
Optuna	0	7 558.047	0.03093	5	0.95576	0.76198
Optuna	42	7 566.117	0.06774	5	0.99831	0.81266
Optuna	7	7 646.793	0.03715	5	0.88691	0.65048
Optuna	123	7 526.869	0.07739	6	0.97856	0.87835
Optuna	999	7 422.765	0.09878	5	0.99347	0.57606
Hyperopt	0	7 662.296	0.06330	6	0.83845	0.76142
Hyperopt	42	7 629.861	0.09793	4	0.92342	0.97096
Hyperopt	7	7 797.638	0.03289	5	0.55027	0.87899
Hyperopt	123	7 645.283	0.04044	6	0.95881	0.75194
Hyperopt	999	7 719.736	0.03937	5	0.89505	0.72961
Skopt	0	7 623.532	0.10000	5	0.80225	1.00000
Skopt	42	7 591.484	0.10000	5	1.00000	0.72552
Skopt	7	7 717.098	0.09310	6	1.00000	0.84858
Skopt	123	7 499.658	0.10000	4	1.00000	1.00000
Skopt	999	7 439.146	0.07717	5	1.00000	0.74533

TABLE X: RQ1 – MLP tuning results across Optuna, Hyperopt and Skopt for five seeds.

Optimizer	Seed	Val RMSE	#Layers	Units	Dropout	LR
Optuna	0	7 048.931	3	222	0.110	0.00447
Optuna	42	6 889.163	2	180	0.061	0.00537
Optuna	7	6 824.281	2	247	0.056	0.00887
Optuna	123	6 722.890	3	59	0.108	0.00401
Optuna	999	6 854.426	3	238	0.010	0.00896
Hyperopt	0	7 086.167	3	173	0.123	0.00231
Hyperopt	42	7 092.317	2	132	0.115	0.00895
Hyperopt	7	7 030.392	3	255	0.217	0.00591
Hyperopt	123	6 721.109	3	178	0.129	0.00987
Hyperopt	999	6 907.978	2	65	0.006	0.00824
Skopt	0	6 876.953	2	256	0.000	0.00535
Skopt	42	7 087.992	3	256	0.000	0.00097
Skopt	7	7 088.376	3	256	0.000	0.00166
Skopt	123	6 967.481	3	32	0.000	0.01000
Skopt	999	6 933.083	3	256	0.000	0.00125